

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Сибирский государственный университет телекоммуникаций и
информатики»

Кафедра телекоммуникационных систем и вычислительных средств
(ТС и ВС)

Отчет по производственной практике
Занятие №14

по теме:

РЕАЛИЗАЦИЯ СИМВОЛЬНОЙ СИНХРОНИЗАЦИИ. ПРИМЕНЕНИЕ К
СЭМПЛАМ С ADALMPLUTO. РАЗРАБОТКА ГРАФИЧЕСКОГО
ИНТЕРФЕЙСА (GUI) ДЛЯ РАБОТЫ С SDR. ОКНА НАСТРОЕК SDR,
ВЫВОД РЕЗУЛЬТАТОВ ОБРАБОТКИ СЭМПЛОВ И Т.Д.

Студент:
Группа ИА-331

Я.А Гмыря

Предподаватели:
Лектор
Семинарист
Семинарист

Калачиков А.А
Ахнашев А.В
Попович И.А

Новосибирск 2025 г.

СОДЕРЖАНИЕ

1	ВВЕДЕНИЕ	3
1.1	Введение	3
1.2	Цель	3
2	ЛЕКЦИЯ	4
2.1	M-th power carrier recovery	4
2.2	Costas loop	5
3	ПРАКТИЧЕСКАЯ ЧАСТЬ	8
3.1	Реализация M-th power carrier recovery	8
3.2	Costas Loop	10
4	ВЫВОД	13

ВВЕДЕНИЕ

1.1 Введение

На этом занятии займемся доработкой GUI для приложения и рассмотрим еще один метод приближенной частотной синхронизации, а также познакомимся с одним из точных методов частотной синхронизации.

1.2 Цель

Доработать GUI. Изучить новые методы частотной синхронизации.

ЛЕКЦИЯ

2.1 M-th power carrier recovery

PSK сигнал можно представить в виде:

$$S(t) = Ae^{i(\omega_c t + \phi_k)}$$

,

где A - амплитуда, ω_c - несущая, ϕ_k - фаза.

Фазу можно представить как $\frac{2\pi k}{M}$, где M - порядок модуляции (сколько точек в созвездии). Если у нас 8-PSK, то мы по сути на окружности расставляем 8 точек с равным интервалом $\frac{\pi}{4}$.

Подставим в сигнал фазу:

$$S(t) = Ae^{i(\omega_c t + \frac{2\pi k}{M})}$$

Предположим, что имеется частотный сдвиг $\Delta\omega_c$ между генератором на передатчике и генератором на приеме и фазовый сдвиг Θ , тогда сигнал на приеме можно записать как:

$$S(t) = Ae^{i(\Delta\omega t + \Theta + \frac{2\pi k}{M})}$$

Теперь делаем следующий фокус: возводим сигнал в степень M :

$$S(t)^M = A^M e^{i(M\Delta\omega t + M\Theta + \frac{2\pi k}{M})}$$

$$S(t)^M = A^M e^{i(M\Delta\omega t + M\Theta + M\frac{2\pi k}{M})}$$

$$S(t)^M = A^M e^{i(M\Delta\omega t + M\Theta)} * e^{i(2\pi k)}$$

Заметим, что $e^{2\pi k}$ всегда равно 1, т.к. $2\pi k$ всегда попадает в точку 0 на окружности. Тогда

$$S(t)^M = A^M e^{i(M\Delta\omega t + M\Theta)}$$

Мы избавились от модуляции и остался только частотный сдвиг $M\Delta\omega$, но мы не знаем, на какой частоте этот сдвиг находится. Для определения частотного сдвига используется FFT. В месте частотного сдвига будет пик.

Когда мы найдем сдвиг, то нам останется только "повернуть" сигнал, домножив его на $e^{-2\pi\Delta f n}$, где n - номер отсчета.

Частоту, найденную в FFT, перед подстановкой в экспоненту нужно разделить на M , т.к мы строим спектр $M\Delta\omega$, а также разделить частоту на sample rate SDR, тогда мы найдем сдвиг, который приходится на семпл.

Алгоритм:

1. Возводим сигнал в степень M .
2. Строим FFT.
3. Ищем пик в FFT и фиксируем его частоту.
4. Делим полученную частоту на M и F_s .
5. Производим восстановление несущей.

Данный метод устранил только большой сдвиг и вряд ли остановит вращение созвездия на приеме. Задача этого алгоритма - подготовить сигнал к точной частотной синхронизации (fine frequency sync). Алгоритм FFS нельзя использовать сразу потому, что он способен восстановить только мелкие сдвиги частоты, а на больших сдвигах работает плохо. Таким образом, два типа алгоритмов частотной синхронизации работают вместе: сначала CFO, потом FFS для более тонкой коррекции.

2.2 Costas loop

Петля Костаса - пример алгоритма точной частотной синхронизации. Работает после символьной синхронизации. Он основывается на созвездии, т.е алгоритму должна быть известна используемая модуляция. Коррекция происходит путем "подтягивания" символа к ближайшей "истинной" точке.

Пусть есть QPSK созвездие, на котором изображены "истинные" точки и одна точка, которую нужно "дотянуть" к "истинной":

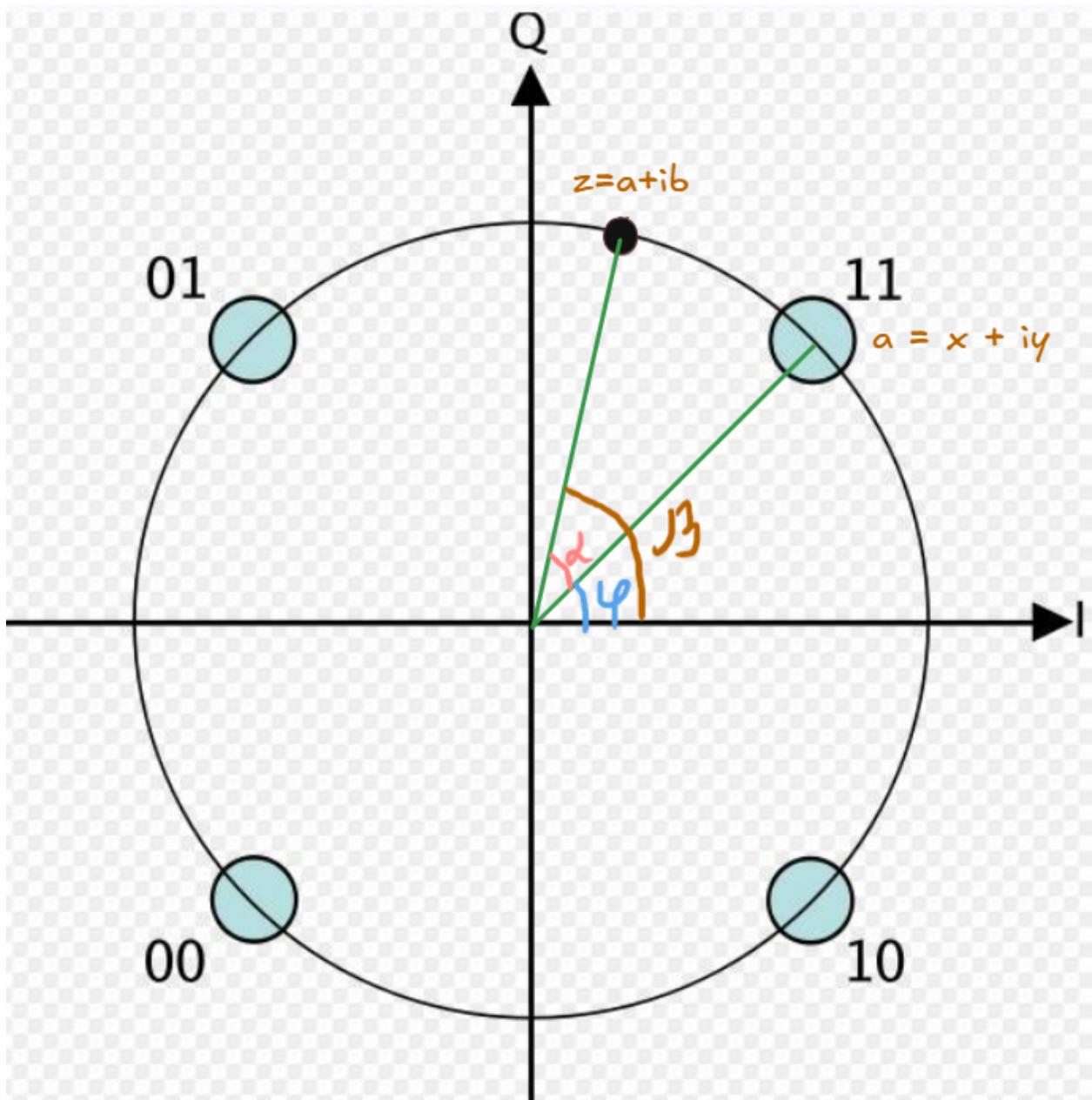


Рисунок 1 — Петля Костаса

Угол α показывает разницу, на которую нужно "провернуть" точку, чтобы она встала на свое место. Задача сводится к нахождению угла α .

Предположим, что сигнал нормированный до $[-1, 1]$, тогда $z = e^{i\beta}$, $a = e^{i\phi}$. Угол α - разница между углами β и ϕ , т.е

$$\alpha = \beta - \phi = e^{i(\beta - \phi)} = z \times a^*$$

В то же время угол α это $\arctan \frac{Imz \times a^*}{Rez \times a^*}$. Arctan - дорогостоящая операция, но его можно аппроксимировать синусом при маленьких значениях. На единичной окружности синус любого угла это просто Im часть, потому что гипотенуза к любой точке на единичной окружности всегда равна 1, остается только противолежащий катет, который равен Im. В итоге получаем:

$$\arctan \frac{Imz}{Rez} = \sin(\alpha) = \sin(z \times a^*) = Im(z \times a^*) = \boxed{y \times a + b \times x}$$

Итоговое выражение сверху - ошибка. Далее ошибка подается на loop filter, где сглаживается и интегрируется.

ПРАКТИЧЕСКАЯ ЧАСТЬ

3.1 Реализация M-th power carrier recovery

```
void synchronizer::coarse_freq_offset(  
    const std::vector<std::complex<double>> &samples, rx_cfg  
        &rx_config,  
    int Fs) {  
  
    std::vector<std::complex<double>>  
        pow_samples(samples.size());  
  
    // pow samples to M-th power  
    for (int i = 0; i < samples.size(); ++i) {  
        pow_samples[i] = std::pow(samples[i],  
            rx_config.mod_order);  
    }  
  
    // calculate FFT  
    rx_config.CFO_spectrum = fft(pow_samples, Fs);  
    int nfft = rx_config.CFO_spectrum.first.size();  
  
    // find FFT peak frequency  
    double max = -__DBL_MAX__;  
    int index;  
  
    for (int i = 0; i < nfft; ++i) {  
        if (std::abs(rx_config.CFO_spectrum.first[i]) > max) {  
            max = std::abs(rx_config.CFO_spectrum.first[i]);  
            index = i;  
        }  
    }  
  
    // calculate frequency offset  
    double df = rx_config.CFO_spectrum.second[index] /  
        rx_config.mod_order;  
    double dw = -2 * M_PI * df / Fs;  
  
    rx_config.post_cfo_signal.resize(rx_config.rx_samples.size());
```



```

// recovery signal
for (int i = 0; i < rx_config.rx_samples.size(); ++i) {
    rx_config.post_cfo_signal[i] =
        samples[i] * std::exp(std::complex<double>(0.0, dw *
            i));
}

// optional: calculate FFT after recovery (check result)
for (int i = 0; i < rx_config.post_cfo_signal.size(); ++i) {
    pow_samples[i] =
        std::pow(rx_config.post_cfo_signal[i],
            rx_config.mod_order);
}

rx_config.post_CFO_spectrum = fft(pow_samples, Fs);
}

```

Здесь структура `rx_config` - структура для межпоточного взаимодействия GUI и основного потока. FFT - дорогая операция, которая требует значительной оптимизации, поэтому я использовал библиотеку `fftw3`.

В результате работы функции у нас должно быть 2 спектра: первый спектр, показывающий частотный сдвиг, и второй спектр, показывающий отсутствие частотного сдвига после восстановления:

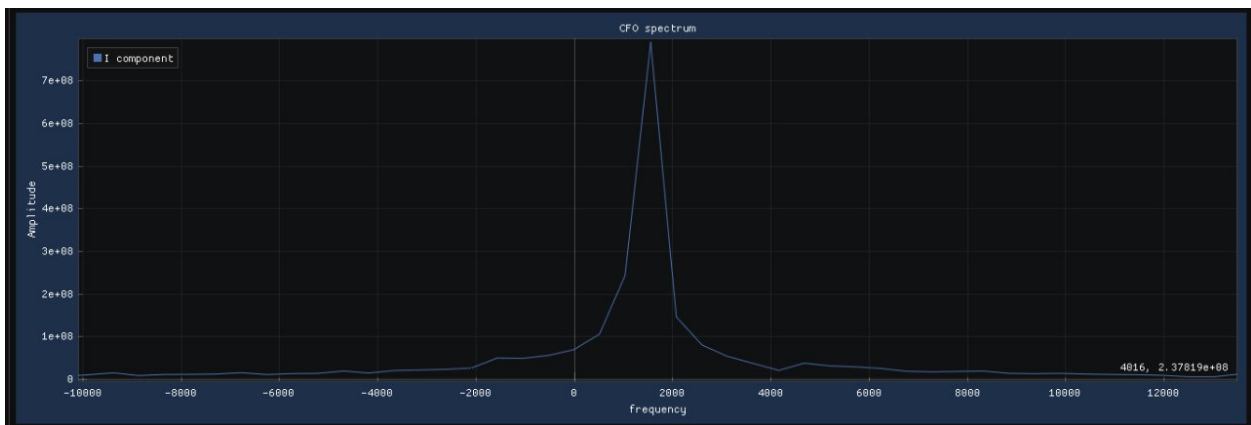


Рисунок 2 — До CFO

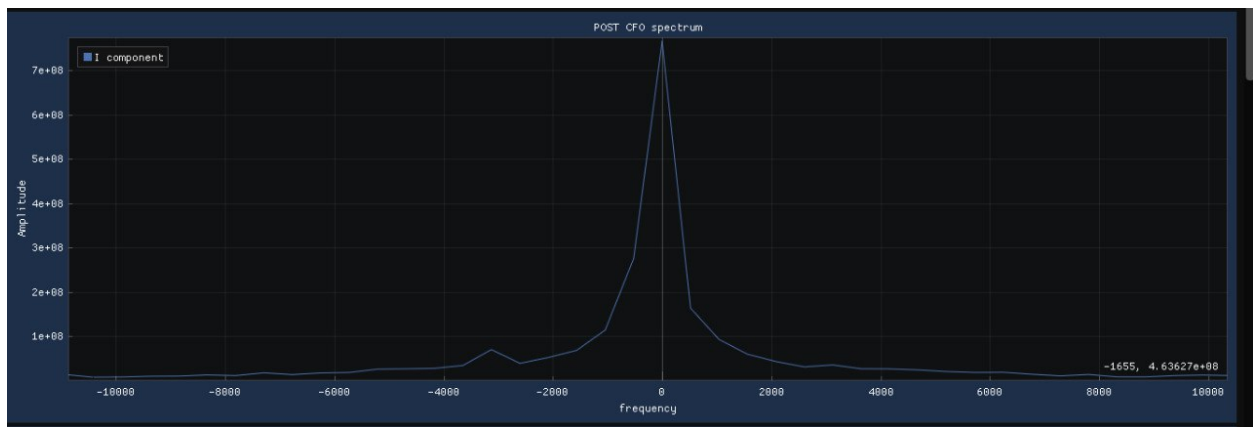


Рисунок 3 — После CFO

На первом изображении частотный сдвиг примерно 1.8кГц, а на втором уже примерно 0. Это означает, что крупный частотный сдвиг устранен, но все равно остается небольшой сдвиг + постоянные колебания этого сдвига, которые заставляют фазу крутиться.

3.2 Costas Loop

```
std::vector<std::complex<double>>
synchronizer::costas_loop(const
    std::vector<std::complex<double>> &samples,
                           const int L, const double BnTs, const
                           double Kp,
                           const int mod_order) {

    // define params for loop filter
    const double zeta = std::sqrt(2) / 2;

    double theta = BnTs / (zeta + 0.25 / zeta);
    double denom = 1 + 2 * zeta * theta + theta * theta;

    double K1 = 4 * zeta * theta / (denom * Kp);
    double K2 = 4 * theta * theta / (denom * Kp);

    double phase = 0.0;
    double freq = 0.0;

    double err;

    std::vector<std::complex<double>> out(samples.size());
```

```

auto sign = [](double x) { return (x > 0) - (x < 0); };

for (size_t i = 0; i < samples.size(); ++i) {
    std::complex<double> rot(std::cos(phase),
        -std::sin(phase));

    std::complex<double> sample = samples[i];

    // rotate
    out[i] = sample * rot;

    double re = out[i].real();
    double im = out[i].imag();

    //compute error
    if (mod_order == 2) {
        double d = (re + im >= 0) ? 1.0 : -1.0;
        err = d * (im - re);
    } else {
        err = sign(re) * im - sign(im) * re;
    }

    //loop filter
    freq += K2 * err;
    phase += freq + K1 * err;
}

return out;
}

```

После символьной синхронизации получаем созвездие, которое очень быстро вращается, образуя при этом окружность. После петли Костаса вращение должно устраниться, а точки должны скопиться в кучки, образуя созвездие.

Пример работы на QPSK:

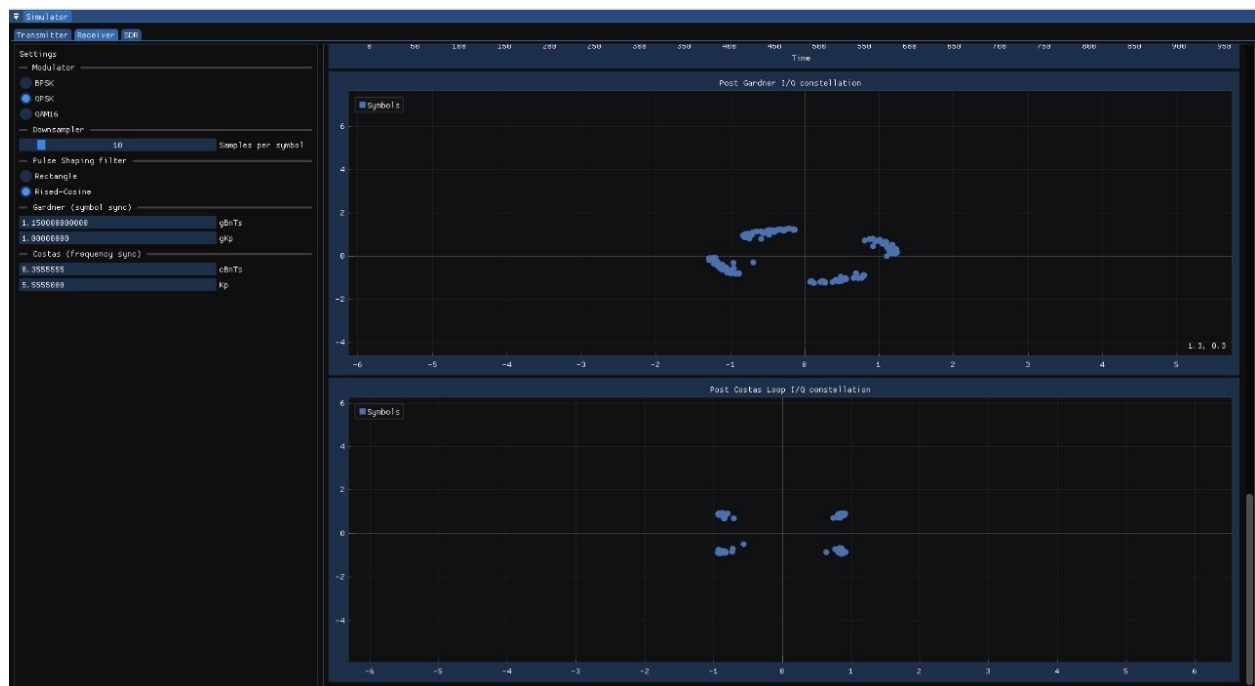


Рисунок 4 — После Costas Loop

На верхнем изображении на самом деле очень быстрое вращение (скриншот не может это передать), а на нижнем рисунке вращения нет и точки собрались в QPSK созвездие.

ВЫВОД

В ходе занятия я познакомился с алгоритмами частотной синхронизации сразу двух видов: coarse и fine, а также на практике реализовал их, что позволило остановить вращение фазы и получить сигнальное созвездие на прямой стороне.